Pedidos online

Supuesto del alumnado

Clases, dataclasses, encapsulación, composición y contratos en Python.

6 etapas	Explicación, desarrollo conjunto y práctica del alumnado.
Duración flexible	El trabajo continúa desde la última etapa comprobada.
Python 3.13	Biblioteca estándar y datos en memoria.
Uso del documento	Reglas, tareas, comprobaciones y reto final sin soluciones.

Cómo consultar este PDF

El índice y los marcadores permiten saltar a cada etapa. Los bloques de código mantienen la sangría y numeran sus líneas; las líneas muy largas continúan visualmente debajo. Los archivos Markdown y Python conservan el texto original para copiarlo.

Escribe el código siguiendo la explicación de clase. La solución no se incluye en este documento.

Índice

Qué funcionará al terminar . . . 3

Preparación . . . 3

Datos y reglas . . . 3

Organización final . . . 4

Datos comunes de las demostraciones . . . 4

Etapa 01 · Clases, instancias y métodos especiales . . . 5

Etapa 02 · Dataclasses y valores que no se reasignan . . . 6

Etapa 03 · Composición y encapsulación del pedido . . . 7

Etapa 04 · Estados y reglas del dominio . . . 8

Etapa 05 · Herencia y contrato nominal con ABC . . . 9

Etapa 06 · Contrato estructural con Protocol e integración . . . 10

Reto final de integración . . . 11

Entrega y autoevaluación . . . 11

Construiremos un pequeño gestor de pedidos de una tienda online. El profesor escribirá una parte del código con la clase y tú resolverás una tarea parecida al final de cada etapa. Conserva el mismo proyecto para que crezca paso a paso.

Qué funcionará al terminar

Podrás crear productos, añadirlos a un pedido, quitar líneas, consultar el total, confirmar y cancelar. El programa emitirá avisos locales por distintos canales. Los datos viven en memoria: cada ejecución empieza de nuevo. No hay menú interactivo, base de datos, correo, pagos, stock ni conexión a Internet.

Preparación

Usa Python 3.13 o superior. Crea pedidos_online, ábrela en VS Code y selecciona su intérprete. Crea tienda y dentro __init__.py. No necesitas instalar paquetes. Ejecuta siempre desde la carpeta que contiene main.py:

Terminal

```
python --version
python -B main.py
```

Escribe los archivos cuando se indique en clase; no crees imports hacia módulos que todavía no existen. Las comprobaciones de cada tarea se pueden ejecutar en un archivo comprobar.py junto a main.py con python -B comprobar.py. Cuando un assert falla, Python muestra AssertionError: revisa ese caso antes de avanzar. No ejecutes con python -O, que desactiva assert.

Datos y reglas

Elemento	Datos y comportamiento
Producto	codigo: str, nombre: str, precio_centimos: int
LineaPedido	producto: Producto, cantidad: int y subtotal calculado
Pedido	numero, líneas propias, total calculado y, desde etapa 04, estado
Notificador	enviar(mensaje: str) -> None, desde etapa 05

- El precio es un entero no negativo: 2500 representa 25 euros. Cantidad: entero positivo. True, False y decimales no se aceptan para precio ni cantidad.
- Código y nombre no pueden estar vacíos. En la primera etapa se limpian espacios; desde la segunda deben llegar ya sin espacios exteriores. El código distingue mayúsculas.
- Cada código solo puede aparecer una vez en un pedido. Añadirlo de nuevo da error; no suma cantidades. Quitar un código inexistente también da error.
- Un pedido nuevo está vacío y suma cero. Dos pedidos no comparten su lista de líneas.
- Se usa ValueError para datos inválidos de construcción y ErrorPedido para operaciones de negocio rechazadas.

Desde etapa 04 aplicamos esta tabla:

Estado actual	Añadir o quitar	Confirmar	Cancelar
ABIERTO	Sí	Solo con líneas	Sí, incluso vacío

Estado actual	Añadir o quitar	Confirmar	Cancelar
CONFIRMADO	Error	Error	Sí
CANCELADO	Error	Error	Error

Cancelar conserva las líneas y el total. Una operación rechazada no cambia nada. numero, lineas, total_centimos y estado se consultan mediante propiedades sin setter. No escribas sobre atributos con _ desde main para saltarte las reglas.

Desde etapa 05 se confirma primero y se avisa después. Si no se puede confirmar, no se envía un aviso. Si el canal falla después de confirmar, el pedido permanece confirmado y el error se propaga.

Organización final

Salida esperada

```
pedidos_online/
  main.py
  tienda/
    __init__.py
    producto.py
    linea.py
    errores.py
    pedido.py
    notificaciones.py
    servicios.py
```

No construiremos toda esta estructura al inicio. Producto aparece en 01, LineaPedido en 02, Pedido y ErrorPedido en 03, los estados en 04 y los avisos en 05. En 06 cambiamos el contrato de avisos.

Datos comunes de las demostraciones

Teclado: código TEC, precio 2500. Ratón: código RAT, precio 1500. Dos teclados y un ratón suman 6500 céntimos. Si quitamos el ratón quedan 5000. Usaremos P001, P002 y P003 como números de ejemplo; no hay registro global que genere o compruebe su unicidad.

Etapa 01 · Clases, instancias y métodos especiales

Construimos juntos

Necesitamos representar un producto y explicar qué diferencia hay entre una plantilla de objetos y cada objeto que creamos.

Trabajaremos estos archivos: `tienda/__init__.py`, `tienda/producto.py`, `main.py`. Sigue la explicación antes de pasar a tu tarea.

Tu tarea

Añade __repr__(self) -> str a Producto. Debe devolver Producto(codigo='TEC', nombre='Teclado', precio_centimos=2500) para los datos del ejemplo. Usa !r para los tres campos y conserva __str__. No cambies el constructor.

Pista si la necesitas: El método devuelve str. Comienza por representar un solo atributo y amplía el f-string. Si aparece None, revisa que estés usando return y no print.

Comprueba tu resultado

Comprobación

```
from tienda.producto import Producto
p = Producto("TEC", "Teclado", 2500)
assert str(p) == "Teclado (2500 céntimos)"
assert repr(p) == "Producto(codigo='TEC', nombre='Teclado', precio_centimos=2500)"
assert p is not Producto("TEC", "Teclado", 2500)
```

Etapa 02 · Dataclasses y valores que no se reasignan

Construimos juntos

El constructor y la representación repiten los campos. Vamos a dejar ese trabajo a dataclass y a modelar una línea con su cantidad.

Trabajaremos estos archivos: `tienda/producto.py`, `main.py`, `tienda/linea.py`. Sigue la explicación antes de pasar a tu tarea.

Tu tarea

Dentro de LineaPedido, escribe la propiedad de solo lectura subtotal_centimos -> int. Su valor es precio_centimos del producto multiplicado por cantidad. Para 2 teclados de 2500 debe dar 5000. No añadas un campo subtotal al dataclass ni un setter.

Pista si la necesitas: Accede primero al producto a través de self.producto y luego a su precio. Añade @property encima del método; al consultarlo no se escriben paréntesis.

Comprueba tu resultado

Comprobación

```
from tienda.linea import LineaPedido
from tienda.producto import Producto
p = Producto("TEC", "Teclado", 2500)
assert p == Producto("TEC", "Teclado", 2500)
assert p is not Producto("TEC", "Teclado", 2500)
assert LineaPedido(p, 2).subtotal_centimos == 5000
```

Etapa 03 · Composición y encapsulación del pedido

Construimos juntos

Un pedido necesita varias líneas y debe controlar cómo se añaden o eliminan. No es una subclase de producto: contiene líneas.

Trabajaremos estos archivos: `main.py`, `tienda/errores.py`, `tienda/pedido.py`. Sigue la explicación antes de pasar a tu tarea.

Tu tarea

Implementa quitar_producto(self, codigo: str) -> None. Busca por el código exacto, elimina la línea correspondiente y termina. Si no existe, lanza ErrorPedido("El producto no está en el pedido") sin modificar nada. No normalices el código ni devuelvas un bool.

Pista si la necesitas: Usa enumerate sobre self._lineas para obtener índice y línea. Después de del, termina con return. El raise de inexistencia debe quedar fuera del bucle.

Comprueba tu resultado

Comprobación

```
from tienda.pedido import Pedido
from tienda.producto import Producto
p = Pedido("P001")
p.agregar_producto(Producto("TEC", "Teclado", 2500), 2)
p.agregar_producto(Producto("RAT", "Ratón", 1500), 1)
assert p.total_centimos == 6500
p.quitar_producto("RAT")
assert p.total_centimos == 5000
assert len(p.lineas) == 1
```

Etapa 04 · Estados y reglas del dominio

Construimos juntos

Los datos solos no evitan operaciones incoherentes: un pedido confirmado debe quedar cerrado a cambios.

Trabajaremos estos archivos: `main.py`, `tienda/pedido.py`. Sigue la explicación antes de pasar a tu tarea.

Tu tarea

Añade cancelar(self) -> None. Puede pasar de ABIERTO o CONFIRMADO a CANCELADO. Si ya está CANCELADO lanza ErrorPedido("El pedido ya está CANCELADO"). Conserva líneas y total. No llames a _exigir_abierto: impediría cancelar un confirmado.

Pista si la necesitas: Solo hay un estado que prohíbe cancelar. Compruébalo antes de asignar el nuevo valor. No borres las líneas.

Comprueba tu resultado

Comprobación

```
from tienda.pedido import Pedido
from tienda.producto import Producto
p = Pedido("P001")
p.agregar_producto(Producto("TEC", "Teclado", 2500), 2)
p.confirmar()
p.cancelar()
assert p.estado == "CANCELADO"
assert p.total_centimos == 5000
assert len(p.lineas) == 1
```

Etapa 05 · Herencia y contrato nominal con ABC

Construimos juntos

Queremos confirmar y emitir un aviso sin obligar al flujo a conocer cómo se presenta o conserva el mensaje.

Trabajaremos estos archivos: `main.py`, `tienda/notificaciones.py`, `tienda/servicios.py`. Sigue la explicación antes de pasar a tu tarea.

Tu tarea

Crear NotificadorMemoria(Notificador). En __init__, inicializar self.mensajes: list[str] = []. Enviar debe añadir exactamente el mensaje recibido y devolver None. La lista pertenece a cada instancia. Usar el mismo confirmar_y_avisar sin modificarlo.

Pista si la necesitas: Hereda de Notificador y escribe enviar con la misma firma. Crea la lista dentro de __init__. No uses print ni una lista a nivel de clase.

Comprueba tu resultado

Comprobación

```
from tienda.notificaciones import NotificadorMemoria
from tienda.pedido import Pedido
from tienda.producto import Producto
from tienda.servicios import confirmar_y_avisar
p = Pedido("P002")
p.agregar_producto(Producto("TEC", "Teclado", 2500), 2)
n = NotificadorMemoria()
confirmar_y_avisar(p, n)
assert n.mensajes == ["Pedido P002 confirmado: 5000 céntimos"]
assert NotificadorMemoria().mensajes == []
```

Etapa 06 · Contrato estructural con Protocol e integración

Construimos juntos

Podemos expresar lo que necesita el servicio sin exigir que todos los canales hereden de una clase común.

Trabajaremos estos archivos: `main.py`, `tienda/notificaciones.py`. Sigue la explicación antes de pasar a tu tarea.

Tu tarea

Crear NotificadorPrefijo sin heredar de ninguna clase del proyecto. Recibe prefijo: str en __init__. enviar(self, mensaje: str) -> None muestra "{prefijo}: {mensaje}". Confirma P003 con este canal sin tocar confirmar_y_avisar. Después realiza el reto final del supuesto.

Pista si la necesitas: Reutiliza la firma de enviar. El prefijo se guarda en self. Para integrarlo, importa la nueva clase en main e inyecta una instancia en confirmar_y_avisar.

Comprueba tu resultado

Comprobación

```
from tienda.notificaciones import NotificadorPrefijo
from tienda.pedido import Pedido
from tienda.producto import Producto
from tienda.servicios import confirmar_y_avisar
p = Pedido("P003")
p.agregar_producto(Producto("TEC", "Teclado", 2500), 2)
confirmar_y_avisar(p, NotificadorPrefijo("TIENDA"))
assert p.estado == "CONFIRMADO"
```

Reto final de integración

Trabaja en reto_final.py junto a main.py. No añadas nuevas reglas al dominio.

1. Crea P100 con dos teclados y un ratón. Comprueba 6500.
2. Quita el ratón. Comprueba 5000.
3. Confirma con NotificadorMemoria. Comprueba estado CONFIRMADO y exactamente un mensaje: Pedido P100 confirmado: 5000 céntimos.
4. Intenta añadir el ratón después de confirmar. Captura ErrorPedido y comprueba que siguen las mismas líneas y el mismo total.
5. Intenta confirmar otra vez usando confirmar_y_avisar y el mismo notificador. Debe fallar y la lista debe seguir teniendo un solo aviso.
6. Cancela P100. Debe conservar 5000 y su línea. Intenta cancelar otra vez: debe fallar sin cambiar nada.
7. Crea P101 vacío e intenta confirmarlo con otro NotificadorMemoria. Debe seguir ABIERTO y sin mensajes.
8. Crea P102 con un ratón y confírmalo con NotificadorPrefijo("CLASE"). Debe mostrar CLASE: Pedido P102 confirmado: 1500 céntimos sin modificar servicios.py.

Si te sobra tiempo, explica por escrito qué ocurriría con un canal cuyo enviar lanzara RuntimeError después de confirmar. No programes reintentos ni rollback.

Entrega y autoevaluación

Entrega pedidos_online con sus archivos .py y reto_final.py. Añade respuestas breves en RESPUESTAS.md:

- ¿Qué diferencia hay entre == e is en nuestros productos?
- ¿Qué métodos ha generado dataclass y qué reglas has escrito tú?
- ¿Por qué devolver una tupla protege mejor la lista interna? ¿Qué límite tiene esa protección?
- ¿Por qué Pedido contiene líneas en lugar de heredar de LineaPedido?
- ¿Qué cambia entre el ejemplo ABC y el de Protocol? ¿Qué comprueba Python al ejecutar?
- ¿Qué pasa si el aviso falla después de confirmar?